# Browser Automation with playwright-cli

## Quick start

```
# open new browser
playwright-cli open
# navigate to a page
playwright-cli goto https://playwright.dev
# interact with the page using refs from the snapshot
playwright-cli click e15
playwright-cli type "page.click"
playwright-cli press Enter
# take a screenshot (rarely used, as snapshot is more common)
playwright-cli screenshot
# close the browser
playwright-cli close
```

## Commands

### Core

```
playwright-cli open
# open and navigate right away
playwright-cli open https://example.com/
playwright-cli goto https://playwright.dev
playwright-cli type "search query"
playwright-cli click e3
playwright-cli dblclick e7
# --submit presses Enter after filling the element
playwright-cli fill e5 "user@example.com"  --submit
playwright-cli drag e2 e8
# drop files or data onto an element (from outside the page)
playwright-cli drop e4 --path=./image.png
playwright-cli drop e4 --data="text/plain=hello world"
playwright-cli hover e4
playwright-cli select e9 "option-value"
playwright-cli upload ./document.pdf
playwright-cli check e12
playwright-cli uncheck e12
playwright-cli snapshot
playwright-cli eval "document.title"
playwright-cli eval "el => el.textContent" e5
# get element id, class, or any attribute not visible in the snapshot
playwright-cli eval "el => el.id" e5
playwright-cli eval "el => el.getAttribute('data-testid')" e5
playwright-cli dialog-accept
```

```
playwright-cli dialog-accept "confirmation text"
playwright-cli dialog-dismiss
playwright-cli resize 1920 1080
playwright-cli close
```

### Navigation

```
playwright-cli go-back
playwright-cli go-forward
playwright-cli reload
```

### Keyboard

```
playwright-cli press Enter
playwright-cli press ArrowDown
playwright-cli keydown Shift
playwright-cli keyup Shift
```

### Mouse

```
playwright-cli mousemove 150 300
playwright-cli mousedown
playwright-cli mousedown right
playwright-cli mouseup
playwright-cli mouseup right
playwright-cli mousewheel 0 100
```

### Save as

```
playwright-cli screenshot
playwright-cli screenshot e5
playwright-cli screenshot --filename=page.png
playwright-cli pdf --filename=page.pdf
```

### Tabs

```
playwright-cli tab-list
playwright-cli tab-new
playwright-cli tab-new https://example.com/page
playwright-cli tab-close
playwright-cli tab-close 2
playwright-cli tab-select 0
```

### Storage

```
playwright-cli state-save
playwright-cli state-save auth.json
playwright-cli state-load auth.json

# Cookies
playwright-cli cookie-list
playwright-cli cookie-list --domain=example.com
playwright-cli cookie-get session_id
playwright-cli cookie-set session_id abc123
playwright-cli cookie-set session_id abc123 --domain=example.com --httpOnly --se
playwright-cli cookie-delete session_id
playwright-cli cookie-clear

# LocalStorage
playwright-cli localstorage-list
playwright-cli localstorage-get theme
playwright-cli localstorage-set theme dark
playwright-cli localstorage-delete theme
playwright-cli localstorage-clear

# SessionStorage
playwright-cli sessionstorage-list
playwright-cli sessionstorage-get step
playwright-cli sessionstorage-set step 3
playwright-cli sessionstorage-delete step
playwright-cli sessionstorage-clear
```

### Network

```
playwright-cli route "**/*.jpg" --status=404
playwright-cli route "https://api.example.com/**" --body='{"mock": true}'
playwright-cli route-list
playwright-cli unroute "**/*.jpg"
playwright-cli unroute
```

### DevTools

```
playwright-cli console
playwright-cli console warning
playwright-cli requests
playwright-cli request 5
playwright-cli run-code "async page => await page.context().grantPermissions(['g
playwright-cli run-code --filename=script.js
playwright-cli tracing-start
```

```
playwright-cli tracing-stop
playwright-cli video-start video.webm
playwright-cli video-chapter "Chapter Title" --description="Details" --duration=
playwright-cli video-stop

# launch the dashboard for UI review / design feedback — user annotates the page
playwright-cli show --annotate

# generate a Playwright locator for an element from its ref or selector
playwright-cli generate-locator e5 --raw

# show a persistent highlight overlay for an element, optionally with a custom s
playwright-cli highlight e5
playwright-cli highlight e5 --style="outline: 3px dashed red"
# hide a single element highlight, or all page highlights when no target is give
playwright-cli highlight e5 --hide
playwright-cli highlight --hide
```

## Raw output

The global --raw option strips page status, generated code, and snapshot sections from the output, returning only the result value. Use it to pipe command output into other tools. Commands that don't produce output return nothing.

```
playwright-cli --raw eval "JSON.stringify(performance.timing)" | jq '.loadEventE
playwright-cli --raw eval "JSON.stringify([...document.querySelectorAll('a')].ma
playwright-cli --raw snapshot > before.yml
playwright-cli click e5
playwright-cli --raw snapshot > after.yml
diff before.yml after.yml
TOKEN=$(playwright-cli --raw cookie-get session_id)
playwright-cli --raw localstorage-get theme
```

For structured output wrapping every reply as JSON, pass –json

```
playwright-cli list --json
```

## Open parameters

```
# Use specific browser when creating session
playwright-cli open --browser=chrome
playwright-cli open --browser=firefox
playwright-cli open --browser=webkit
playwright-cli open --browser=msedge
```

```
# Use persistent profile (by default profile is in-memory)
playwright-cli open --persistent
# Use persistent profile with custom directory
playwright-cli open --profile=/path/to/profile

# Connect to browser via Playwright Extension
playwright-cli attach --extension=chrome

# Connect to a running Chrome or Edge by channel name
playwright-cli attach --cdp=chrome
playwright-cli attach --cdp=msedge

# Connect to a running browser via CDP endpoint
playwright-cli attach --cdp=http://localhost:9222

# Start with config file
playwright-cli open --config=my-config.json

# Close the browser
playwright-cli close
# Detach from an attached browser (leaves the external browser running)
playwright-cli -s=msedge detach
# Delete user data for the default session
playwright-cli delete-data
```

## Snapshots

After each command, playwright-cli provides a snapshot of the current browser state.

```
> playwright-cli goto https://example.com
### Page
- Page URL: https://example.com/
- Page Title: Example Domain
### Snapshot
[Snapshot](.playwright-cli/page-2026-02-14T19-22-42-679Z.yml)
```

You can also take a snapshot on demand using `playwright-cli snapshot` command. All the options below can be combined as needed.

```
# default - save to a file with timestamp-based name
playwright-cli snapshot

# save to file, use when snapshot is a part of the workflow result
playwright-cli snapshot --filename=after-click.yaml
```

```
# snapshot an element instead of the whole page
playwright-cli snapshot "#main"

# limit snapshot depth for efficiency, take a partial snapshot afterwards
playwright-cli snapshot --depth=4
playwright-cli snapshot e34

# include each element's bounding box as [box=x,y,width,height]
playwright-cli snapshot --boxes
```

### Targeting elements

By default, use refs from the snapshot to interact with page elements.

```
# get snapshot with refs
playwright-cli snapshot

# interact using a ref
playwright-cli click e15
```

You can also use css selectors or Playwright locators.

```
# css selector
playwright-cli click "#main > button.submit"

# role locator
playwright-cli click "getByRole('button', { name: 'Submit' })"

# test id
playwright-cli click "getByTestId('submit-button')"
```

### Browser Sessions

```
# create new browser session named "mysession" with persistent profile
playwright-cli -s=mysession open example.com --persistent
# same with manually specified profile directory (use when requested explicitly)
playwright-cli -s=mysession open example.com --profile=/path/to/profile
playwright-cli -s=mysession click e6
playwright-cli -s=mysession close  # stop a named browser
playwright-cli -s=mysession delete-data  # delete user data for persistent sessi

playwright-cli list
# Close all browsers
playwright-cli close-all
# Forcefully kill all browser processes
playwright-cli kill-all
```

## Installation

If global `playwright-cli` command is not available, try a local version via `npx playwright-cli`:

```
npx --no-install playwright-cli --version
```

When local version is available, use `npx playwright-cli` in all commands. Otherwise, install `playwright-cli` as a global command:

```
npm install -g @playwright/cli@latest
```

## Example: Form submission

```
playwright-cli open https://example.com/form
playwright-cli snapshot

playwright-cli fill e1 "user@example.com"
playwright-cli fill e2 "password123"
playwright-cli click e3
playwright-cli snapshot
playwright-cli close
```

## Example: Multi-tab workflow

```
playwright-cli open https://example.com
playwright-cli tab-new https://example.com/other
playwright-cli tab-list
playwright-cli tab-select 0
playwright-cli snapshot
playwright-cli close
```

## Example: Debugging with DevTools

```
playwright-cli open https://example.com
playwright-cli click e4
playwright-cli fill e7 "test"
playwright-cli console
playwright-cli requests
playwright-cli close

playwright-cli open https://example.com
playwright-cli tracing-start
playwright-cli click e4
playwright-cli fill e7 "test"
playwright-cli tracing-stop
playwright-cli close
```

## Example: Interactive session

Ask the user for UI review or design feedback. The user draws boxes on the live page and types comments; you receive the annotated screenshot, the snapshot of the marked region, and the user's notes. Use this whenever the user asks for "UI review", "design feedback", or to "ask the user what they think / want / mean":

```
playwright-cli open https://example.com
playwright-cli show --annotate
```

## Specific tasks

- **Running and Debugging Playwright tests** references/playwright-tests.md
- **Request mocking** references/request-mocking.md
- **Running Playwright code** references/running-code.md
- **Browser session management** references/session-management.md
- **Spec-driven testing (plan / generate / heal)** references/spec-driven-testing.md
- **Storage state (cookies, localStorage)** references/storage-state.md
- **Test generation** references/test-generation.md
- **Tracing** references/tracing.md
- **Video recording** references/video-recording.md
- **Inspecting element attributes** references/element-attributes.md